

기능 설명서

login_test

Flask + Supabase 로 만든 회원 인증 웹사이트

[회원가입](#) · [로그인](#) · [비밀번호 관리](#) · [관리자 콘솔](#)

[Python](#) / [Flask](#) / [Supabase\(PostgreSQL\)](#) / [Vercel](#)

OVERVIEW

이 사이트는 무엇인가요?

회원이 가입하고 로그인할 수 있는 웹사이트입니다. 관리자는 별도의 콘솔에서 회원을 관리하고 사이트 설정을 바꿀 수 있습니다.

구성 요소	역할
Flask (app.py)	화면을 보여주고 사용자의 요청을 처리하는 서버
Supabase (PostgreSQL)	회원 정보 · 로그인 기록을 저장하는 데이터베이스
templates/	사용자에게 보이는 HTML 화면
Vercel	사이트를 인터넷에 공개하는 배포 플랫폼

데이터가 오가는 흐름

브라우저 → Flask → Supabase 함수 → 결과 반환

비밀번호는 Flask 를 그냥 거쳐 지나갈 뿐이고, 실제 암호화와 검증은 모두 Supabase 데이터베이스 안에서 이루어집니다. 덕분에 비밀번호 원문은 물론 암호화된 값조차 서버 바깥으로 나오지 않습니다.

데이터베이스 테이블

테이블	저장하는 것
users	아이디, 암호화된 비밀번호, 이메일, 활성 여부, 관리자 여부, 가입일
login_attempts	모든 로그인 시도 (아이디, IP, 성공 여부, 시각)
password_resets	비밀번호 재설정 토큰 (해시만 보관, 1시간 후 만료)
app_settings	회원가입 on/off 상태, 서버 전용 비밀키

일반 회원이 쓰는 기능

회원가입 /signup

아이디(3자 이상), 비밀번호(6자 이상), 이메일(선택)을 입력해 계정을 만듭니다. 아이디 중복, 비밀번호 길이, 비밀번호 확인 일치 여부를 모두 검사합니다. 관리자가 회원가입을 꺼두면 이 페이지는 안내 문구만 표시되고, 폼을 우회한 직접 요청도 서버에서 차단됩니다.

로그인 /login

아이디와 비밀번호로 로그인합니다. 로그인 상태 유지를 체크하면 7일간 세션이 유지됩니다. 아이디가 없을 때와 비밀번호가 틀릴 때 똑같은 메시지를 보여주어, 어떤 아이디가 존재하는지 알아낼 수 없게 했습니다.

비밀번호 변경 /change-password

로그인한 상태에서 현재 비밀번호를 확인한 뒤 새 비밀번호로 바꿉니다. 현재 비밀번호를 모르면 변경할 수 없습니다.

비밀번호 찾기 /forgot-password

아이디를 입력하면 1시간 동안만 유효한 재설정 링크가 발급됩니다. 이메일 발송이 설정되어 있으면 메일로 전송되고, 설정되어 있지 않으면 관리자가 콘솔에서 링크를 만들어 전달합니다. 존재하지 않는 아이디를 넣어도 똑같은 안내가 나와 계정 존재 여부가 새지 않습니다.

회원 탈퇴 /delete-account

비밀번호를 다시 확인한 뒤 계정을 완전히 삭제합니다. 되돌릴 수 없습니다.

관리자 콘솔

관리자 계정으로 로그인하면 일반 대시보드 대신 관리자 콘솔로 이동합니다. 상단 메뉴로 네 개의 화면을 오갈 수 있습니다.

화면	할 수 있는 일
대시보드 /admin	회원 수 · 로그인 통계 요약, 회원가입 기능 on/off 전환, 대기 중인 비밀번호 재설정 요청 확인
회원 관리 /admin/users	회원 목록 조회(검색 · 페이지네이션), 계정 정지/활성화, 관리자 권한 부여/해제, 재설정 링크 발급, 회원 삭제
로그인 기록 /admin/logs	모든 로그인 시도 기록(시각 · 아이디 · IP · 성공 여부) 확인, 잠긴 계정 수동 해제
통계 /admin/stats	최근 30일 일별 가입자 수 막대 그래프, 전체/활성/관리자 회원 수

관리자가 되는 두 가지 방법

환경변수 관리자 — ADMIN_USERNAME / ADMIN_PASSWORD 로 지정한 계정. 데이터베이스와 무관하게 항상 로그인할 수 있어, 관리자 계정을 잃어버려도 사이트에서 잠기지 않는 비상용 통로입니다.

데이터베이스 관리자 — 회원 관리 화면에서 일반 회원에게 관리자로 버튼을 누르면 그 계정이 관리자가 됩니다. 코드를 고치지 않고도 관리자를 늘리거나 줄일 수 있습니다.

보안은 이렇게 지킵니다

비밀번호는 절대 원문으로 저장하지 않습니다

PostgreSQL 의 `bcrypt(crypt + gen_salt)` 로 단방향 암호화해 저장합니다. 되돌릴 수 없기 때문에 데이터베이스가 통째로 유출되어도 비밀번호는 안전합니다. 로그인 시에는 입력값을 같은 방식으로 암호화해서 비교할 뿐, 복호화하지 않습니다.

세션 위조를 막습니다

로그인 상태는 서명된 쿠키로 관리하며, 서명에 쓰는 `SECRET_KEY` 는 코드가 아닌 환경변수에 둡니다. 쿠키에는 `HttpOnly`(자바스크립트 접근 차단), `SameSite=Lax`, 배포 환경에서는 `Secure`(HTTPS 전용) 옵션을 적용했습니다.

무차별 대입을 차단합니다

15분 안에 5회 로그인에 실패하면 해당 아이디가 자동으로 잠깁니다. 이 판정은 서버 메모리가 아니라 데이터베이스에서 이루어지므로, Vercel 처럼 서버가 여러 개로 나뉘어 실행되는 환경에서도 정확히 동작합니다.

CSRF 공격을 막습니다

모든 폼에 일회용 토큰(`csrf_token`)을 넣었습니다. 관리자가 로그인한 상태로 악성 사이트를 방문해도, 본인 모르게 회원이 삭제되거나 설정이 바뀌는 일을 막습니다.

데이터베이스를 직접 열지 못하게 합니다

모든 테이블에 RLS(행 수준 보안)를 켜고 정책을 하나도 만들지 않아, 공개 키로는 테이블을 직접 조회할 수 없습니다. 데이터 접근은 오직 검증 로직이 들어 있는 함수를 통해서만 가능합니다.

공개 키가 유출되어도 관리자 기능은 안전합니다

회원 삭제 같은 함수는 서버만 아는 비밀키(`SUPABASE_RPC_SECRET`)를 함께 보내야만 동작합니다. Supabase 의 `publishable` 키는 공개되어도 되는 키이므로, 이 이중 잠금이 없으면 누구나 관리자 함수를 호출할 수 있습니다.

계정 존재 여부를 노출하지 않습니다

로그인 실패와 비밀번호 찾기 모두, 계정이 있든 없든 동일한 메시지를 보여줍니다. 공격자가 유효한 아이디 목록을 수집하는 것을 막기 위해서입니다.

SETUP

설치와 환경변수

필요한 환경변수

이름	설명	필수
SUPABASE_URL	Supabase REST 엔드포인트 주소	예
SUPABASE_KEY	publishable(anon) 키	예
SUPABASE_RPC_SECRET	SQL 스크립트 실행 시 출력되는 서버 전용 비밀키	예
SECRET_KEY	세션 서명용 무작위 문자열	예
ADMIN_USERNAME	비상용 관리자 아이디	권장
ADMIN_PASSWORD	비상용 관리자 비밀번호	권장
RESEND_API_KEY	비밀번호 재설정 메일 발송용 (없으면 수동 전달)	선택

처음 설치하는 순서

Supabase 대시보드 → SQL Editor 에서 supabase_setup.sql 전체를 실행합니다.

실행 결과 맨 아래에 출력되는 UUID 를 복사해 SUPABASE_RPC_SECRET 에 넣습니다.

SECRET_KEY 를 생성합니다 (아래 명령).

로컬은 .env 파일에, 배포는 Vercel → Settings → Environment Variables 에 값을 넣습니다.

환경변수를 바꾼 뒤에는 반드시 재배포해야 반영됩니다.

```
# SECRET_KEY 생성
python -c "import secrets; print(secrets.token_hex(32))"
# 로컬 실행
pip install -r requirements.txt
python app.py
```

주의할 점

.env 파일은 절대 Git 에 올리지 마세요. .gitignore 에 등록되어 있습니다.

SECRET_KEY 를 바꾸면 기존 로그인 세션이 모두 풀립니다 (정상 동작입니다).

SUPABASE_RPC_SECRET 이 틀리면 모든 기능이 실패합니다. 값을 정확히 복사하세요.

회원가입을 꺼두어도 기존 회원의 로그인은 정상 동작합니다.

이 문서는 login_test 프로젝트의 기능 설명서입니다. 코드 자체에 대한 자세한 해설은 별도 문서를 참고하세요.